Отчёт. Векторизация КТ-данных грецкого ореха: сегментация срезов и построение 3D-моделей

Студент: Леликов Слободан
3 курс, группа: БПМ-23-ПО-3
Проверил: доцент, Полевой Д. В.
Github: ssldeaf/misis2026s-23-03-lelikov-s-s

Содержание

1. Постановка задачи.....3
2. Данные и целевые классы.....4
3. Сегментация: архитектура пайплайна (segm.py).....7
3.1. Стадия A: маска ореха (build_walnut_mask).....8
3.2. Стадия B: скорлупа и внутренность (build_shell).....12
3.3. Стадия C: ядро (build_kernel).....14
3.4. Стадия D: перегородки (build_partitions).....18
3.5. Итоговый overlay среза.....20
4. Оценка точности сегментации.....22
4.1. Метрики Dice и IoU.....22
4.2. Контроль площадей по объёму (area_check.py).....22
4.3. Диагностика сегментации по ортогональным проекциям (xz/yz).....23
5. Построение 3D-мешей (build_mesh.py).....25
5.1. Декодирование overlay в метки (decode_labels).....25
5.2. Базовый метод и первые артефакты.....25
5.3. Отладка: очистка и устранение артефактов.....26
5.4. Итоговые меши.....29
6. Результаты.....31

1. Постановка задачи

Дан КТ-скан грецкого ореха (датасет Walnut2), состоящий из 501 среза в формате TIFF. Конечная цель работы — получить связное трёхмерное представление ореха, разделённое по анатомическим классам: скорлупа, ядро и перегородки. Работа разбита на два больших этапа, выполняемых последовательно.

Этап 1 — сегментация (segm.py)

Вход этапа — отдельный КТ-срез (8-битное изображение после нормализации). Выход — для каждого среза три бинарные маски (скорлупа, ядро, перегородки) и контрольный цветной overlay. Этап решает задачу классификации каждого пикселя среза в один из четырёх классов (фон / скорлупа / ядро / перегородка) на основе яркости, формы и взаимного расположения структур. Алгоритм работает посрезово и параметрически: числовые пороги, чувствительные к положению среза вдоль ореха, интерполируются по номеру среза (si). Этап реализован в segm.py и состоит из четырёх последовательных стадий $A \rightarrow B \rightarrow C \rightarrow D$ (раздел 3), где маска каждой стадии ограничивает область поиска следующей. Результат всех срезов — стопка overlay-изображений, которая служит входом второго этапа.

Этап 2 — векторизация и построение 3D-мешей (build_mesh.py)

Вход этапа — стопка overlay-изображений (результат этапа 1), сложенная в трёхмерный воксельный массив. Выход — три полигональные поверхности (mesh) классов в форматах OBJ и PLY. Этап решает задачу восстановления непрерывной 3D-поверхности из дискретного набора 2D-масок: цвета overlay декодируются обратно в метки классов, по каждому классу строится изоповерхность методом Marching Cubes, после чего поверхность очищается от артефактов (мелкие компоненты, обрезанные края, паразитные заполнения) и приводится к виду, пригодному для дальнейшей работы с сетками. Этап реализован в build_mesh.py (раздел 5).

2. Данные и целевые классы

Исходные данные — стопка из 501 среза формата TIFF (16-битные значения плотности рентгеновского поглощения). Орех присутствует не на всех срезах: рабочий объём — примерно срезы 67–433. Автоматическим пайплайном segm.py сегментирована основная, центральная часть этого диапазона. Маски ранних (≈67–100) и поздних (≈400–433) срезов построены вручную: на них ткань ореха выражена слабо и нетипично, автоматическая сегментация ненадёжна, поэтому эти срезы были размечены вручную.

Прежде чем переходить к алгоритму, зафиксируем, что именно требуется получить. Ниже на одном срезе (si = 303) показаны исходное изображение ореха и три целевые маски-класса, которые должен выдать этап сегментации.

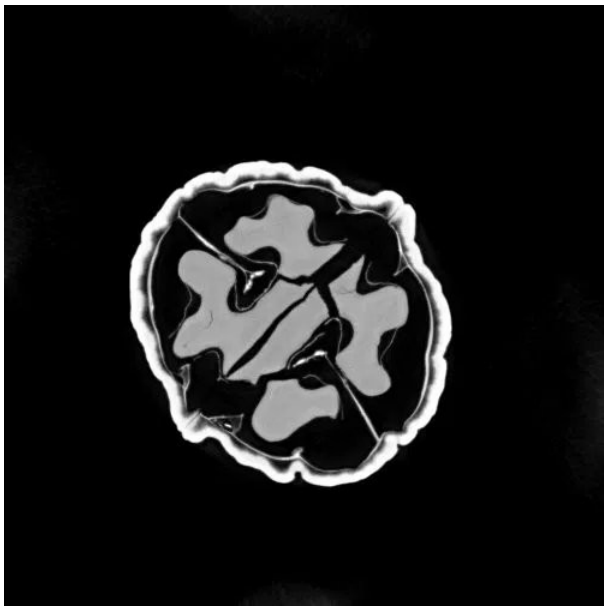


Рис. 1. Исходный КТ-срез ореха (si = 303): яркая скорлупа-кольцо, тёмные полости, ядро и тонкие перегородки.

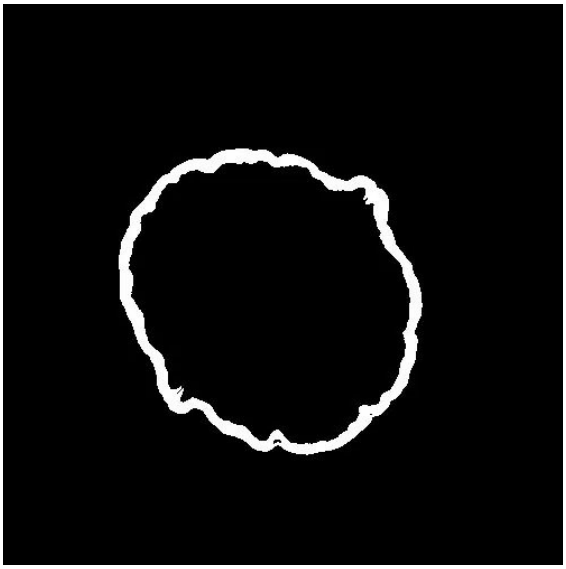


Рис. 2. Целевая маска класса «скорлупа» (shell) — внешняя оболочка ореха.

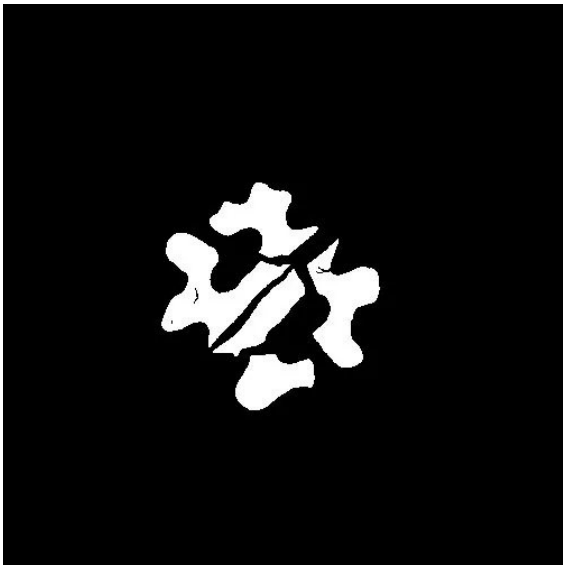


Рис. 3. Целевая маска класса «ядро» (kernel) — лопасти съедобной части.

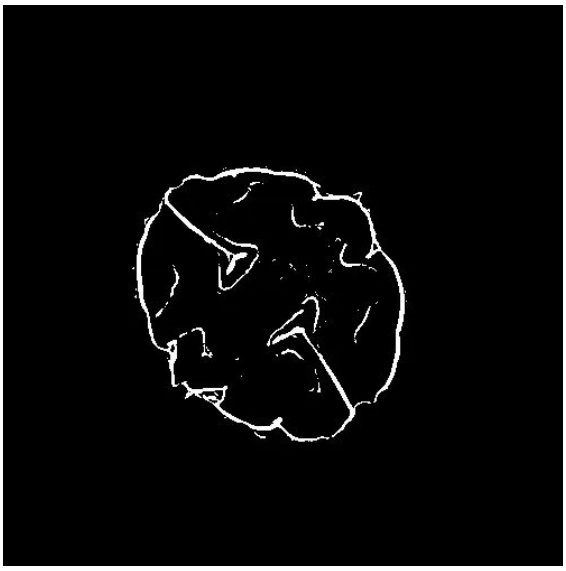


Рис. 4. Целевая маска класса «перегородки» (septa) — тонкие перегородки между лопастями.

Задача этапа 1 — построить эти три маски автоматически для каждого среза. Ниже описано, как load_slice готовит изображение, после чего пайплайн стадий A–D последовательно выделяет классы.

Загрузка и нормализация одного среза выполняется функцией load_slice(path):

```
def load_slice(path):
    raw = tifffile.imread(path).astype(np.float32)
    p_high = np.percentile(raw, 99)     # верхний 99-й перцентиль
    raw = np.clip(raw, None, p_high)    # гасим сверхъяркие, повышаем контраст
    img = cv2.normalize(raw, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
    # si извлекается из имени файла регулярным выражением
    return img, img, si
```

Клиппирование по верхнему 99-му перцентилю гасит единичные сверхъяркие пиксели и повышает контраст: после него нормализация растягивает в 0–255 именно полезный диапазон яркостей ореха, не «съедаемый» редкими выбросами. Нижний порог не применяется — изображение полутоновое, его тёмная часть это фон, обрезать её не требуется. Результат — 8-битное изображение, готовое к сегментации (Рис. 5).

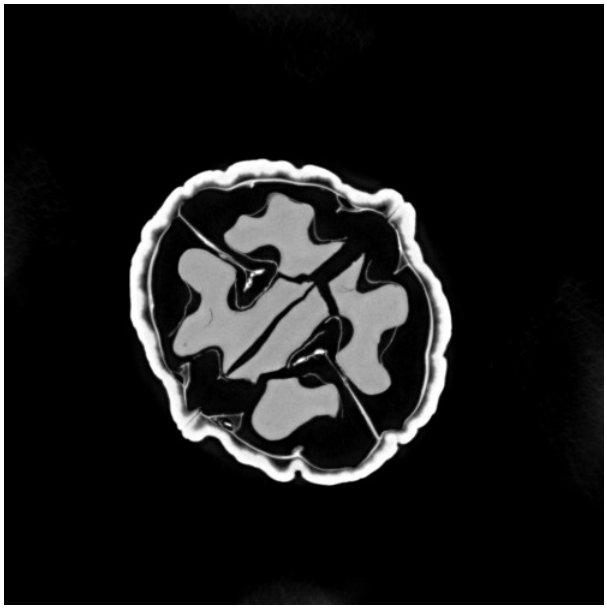


Рис. 5. Срез si = 303 после загрузки, клиппирования и нормализации в диапазон 0–255.

3. Сегментация: архитектура пайплайна (segm.py)

Пайплайн разбит на четыре последовательные стадии. Маски, найденные на предыдущей стадии, используются как ограничивающие маски или априорную инициализацию для следующей:

- **A** build_walnut_mask — построение маски всего ореха;
- **B** build_shell — отделение скорлупы от внутренней части (interior);
- **C** build_kernel — выделение ядра внутри interior;
- **D** build_partitions — выделение перегородок между лопастями ядра и в зоне скорлупы.

Параметризация по срезу

Все числовые параметры, чувствительные к номеру среза, вынесены в словарь PARAM_ANCHORS и интерполируются функцией interp_param(name, si). Словарь хранит значения в виде пар (si, value); функция сортирует анкоры и возвращает линейно проинтерполированное значение между двумя соседними. Если si меньше минимального или больше максимального анкора — берётся значение крайнего.

```
def interp_param(name, si):
    anchors = sorted(PARAM_ANCHORS[name])
    if si <= anchors[0][0]:  return anchors[0][1]
    if si >= anchors[-1][0]: return anchors[-1][1]
    for (xa, pa), (xb, pb) in zip(anchors[:-1], anchors[1:]):
        if xa <= si <= xb:
            return pa + (pb - pa) * (si - xa) / (xb - xa)
```

Такой подход позволяет точечно настраивать алгоритм для отдельных диапазонов срезов, не трогая логику. Структурные параметры, не зависящие от si (размеры морфологических ядер, пороги площадей), оставлены отдельными константами.

Полный перечень параметров пайплайна приведён ниже. Якорные (si-зависимые) параметры хранятся в PARAM_ANCHORS и интерполируются по номеру среза; структурные параметры от si не зависят и заданы отдельными константами.

Параметр	Стадия	Назначение
shell_pct	B	Перцентиль яркости в кромке маски для порога скорлупы
seed_pct	C	Перцентиль яркости для выбора «семян» region growing (ядро)
area_frac	C	Минимальная площадь лопасти ядра как доля площади interior (порог min_lobe)
solidity_min	C	Мин. плотность формы (площадь / выпуклая оболочка) — фильтр ядра
aspect_min	C	Мин. вытянутость (малая/большая ось эллипса) — фильтр ядра
frangi_pct	D	Перцентиль отклика Frangi для порога центральных перегородок
ring_bright_pct	D	Перцентиль яркости в кольце вокруг ядра для «мостиков»-перегородок
bright_filter_pct	D	Перцентиль яркости внутри компоненты-мостика (оставляем только ярче порога)

Таблица 1. Якорные (si-зависимые) параметры — словарь PARAM_ANCHORS.

Параметр	Значение	Назначение
TOLERANCE	20	Допуск по яркости при region growing (стадия C)
HOLE_FRAC_MAX	0.20	Макс. доля внутренних дыр в компоненте ядра
KERNEL_MIN_SI / MAX_SI	70 / 383	Диапазон срезов, где ищется ядро (вне его — пустая маска)
FRANGI_SIGMAS	1–3	Масштабы (σ) фильтра Frangi для перегородок
FRANGI_INTERIOR_ERODE	3	Эрозия interior перед Frangi (убрать краевую полосу)
FRANGI_KERNEL_DILATE	1	Расширение ядра перед вычитанием из перегородок
RING_PX	15	Ширина кольца вокруг ядра (px) для поиска мостиков
KERNEL_EXPAND_PX	2	Расширение ядра перед яркостным фильтром мостиков
SHELL_ZONE_DILATE_PX	3	Расширение скорлупы при вырезании зоны скорлупы
SHELL_ZONE_BRIGHT_T	50	Абсолютный порог яркости перегородок в зоне скорлупы
SHELL_ZONE_MIN/MAX_AREA	100 / 1000	Диапазон площади компонент перегородок в зоне скорлупы

Таблица 2. Фиксированные (структурные) параметры, не зависящие от si.

Значения анкоров подобраны эмпирически. По 42 эталонным маскам (§4.1) и debug-кадрам для опорных срезов параметр настраивался так, чтобы результат совпадал с эталоном или визуально верной маской; найденная пара (si, значение) добавлялась в PARAM_ANCHORS. Плотность анкоров неравномерна — она выше там, где структура ореха быстро меняется вдоль оси z.
Между опорными точками значение восстанавливается линейной интерполяцией (функция interp_param): для si между соседними анкорами (xa, pa) и (xb, pb) возвращается pa + (pb − pa)·(si − xa)/(xb − xa); вне крайних анкоров берётся значение ближайшего из них. Линейная схема выбрана как простейшая непрерывная: она даёт плавное изменение параметра по объёму и не требует правок кода — для подстройки достаточно добавить новую точку (si, значение).

Debug-механизм

Функция process_slice(path, debug, output_dir) оркеструет все стадии. При debug=True каждой стадии передаётся коллбэк save(name, image), который сохраняет промежуточный результат под именем NNN_func__step_siXXX.png (NNN — сквозной счётчик вызовов). Так каждая морфологическая операция фиксируется отдельным изображением, а их порядок воспроизводим. Команда запуска:

```
python segm.py --mode debug      # один срез, со всеми промежуточными шагами
```

Прогон на срезе 303 дал 48 пронумерованных изображений. Ниже показаны ключевые из них по каждой стадии.

3.1. Стадия A: маска ореха (build_walnut_mask)

Функция строит бинарную маску всего ореха. Ниже каждый шаг показан и обоснован — в том числе наложением на исходное изображение и сравнением с эталоном (числами).

Шаг 1 — порог OTSU и зачем он нужен

Скорлупа на КТ заметно ярче фона и большей части внутренних полостей. Метод OTSU (cv2.THRESH_OTSU) автоматически подбирает порог, максимизирующий межклассовую дисперсию яркостей — то есть сам находит границу «тёмное/светлое» без ручного задания значения. Результат бинаризации показан на Рис. 6, а на Рис. 7 он наложен зелёным на исходный срез: видно, что OTSU уверенно захватывает яркое кольцо скорлупы и светлые фрагменты, но оставляет тёмные внутренние полости незаполненными (это и есть причина последующих шагов).

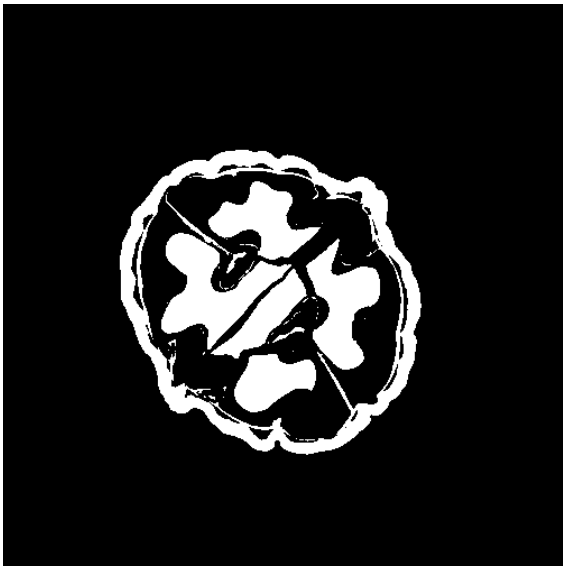


Рис. 6. Стадия А, шаг 1: бинаризация методом OTSU.

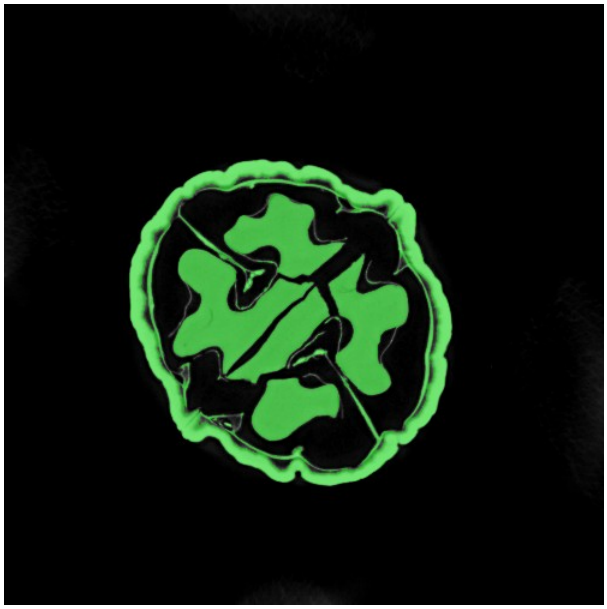


Рис. 7. Полезность OTSU: результат порога (зелёный) наложен на исходный срез. Захвачено 28 781 px — яркая скорлупа и светлые участки; тёмные полости внутри пока вне маски.

Шаг 2 — морфологическое закрытие и заливка контура

Неотфильтрованный результат OTSU содержит крупные внутренние пропуски: тёмные полости и перегородки внутри ореха порог не захватил. Но для маски ореха требуется сплошной силуэт. Морфологическое закрытие ядром 7×7 (дилатация, затем эрозия) смыкает разрывы, после чего заливка наибольшего контура (шаг 3) закрывает

оставшиеся внутренние полости целиком. На Рис. 8 показан результат закрытия, а на Рис. 9 — наглядно, что именно добавляют закрытие и заливка: зелёным исходный OTSU, жёлтым — восстановленные внутренние области.

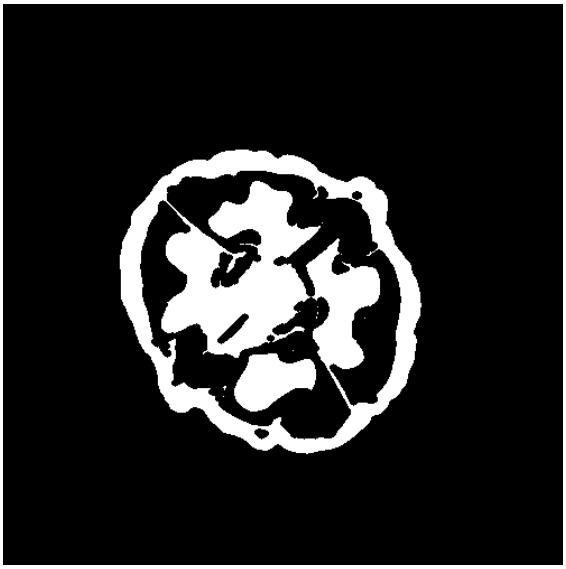


Рис. 8. Стадия A, шаг 2: результат морфологического закрытия.

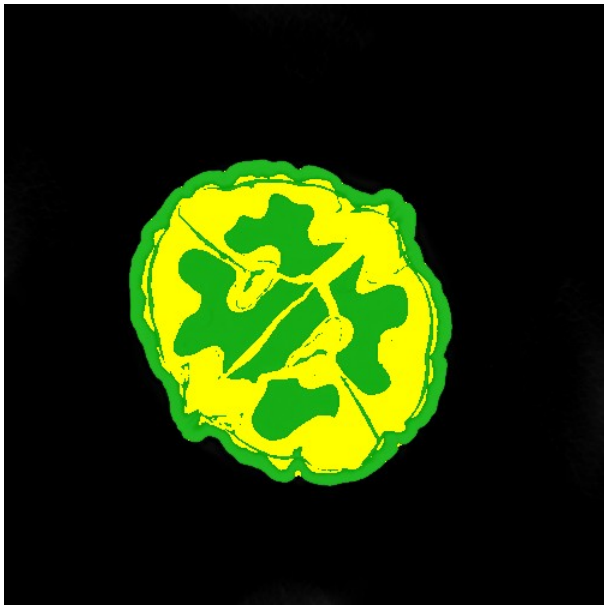


Рис. 9. Смысл морфологии: зелёным — что нашёл OTSU (28 781 px), жёлтым — внутренние полости (≈ 27 100 px), восстановленные закрытием и заливкой контура. Без этого шага маска ореха была бы «дырявой».

Шаг 3 — выбор наибольшего центрального контура

Среди всех внешних контуров (cv2.findContours) отбираются прошедшие два фильтра: площадь не меньше min_area = 5000 px (отсекает мелкий шум) и центр масс не дальше 0.4·min(H,W) от центра кадра (отсекает посторонние объекты у краёв). Из прошедших выбирается наибольший и заливается сплошным белым (Рис. 10). Эта залитая маска уже совпадает с эталонным силуэтом ореха на Dice = 0.982 (IoU = 0.964).

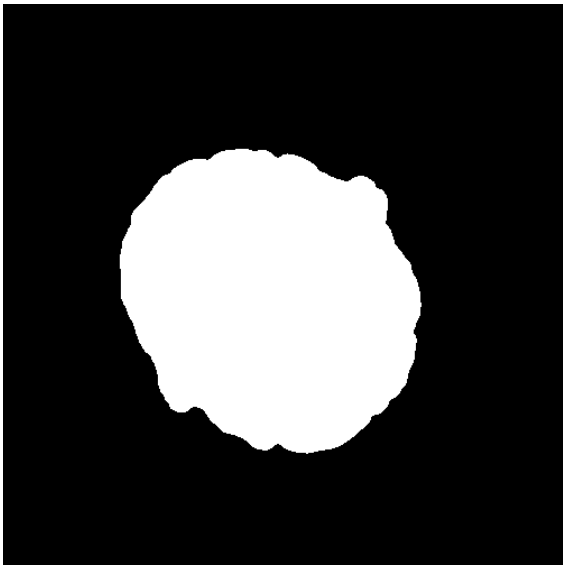


Рис. 10. Стадия A, шаг 3: выбран и залит наибольший центральный контур.

Шаг 4 — дилатация и контроль по эталону

Заливка контура не доходит до внешней кромки скорлупы (примерно на 1 пиксель). Лёгкая дилатация ядром 3×3 расширяет маску на ~1 пиксель наружу, гарантированно захватывая край скорлупы. Эффект небольшой по площади (+806 px), но повышает согласие с эталоном: Dice растёт с 0.982 до 0.989 (IoU 0.964 → 0.978). На Рис. 11 красным показано кольцо, добавленное дилатацией, на Рис. 12 — сравнение итоговой маски с эталоном (зелёный — совпадение, синий — недосегментация 1267 px, красный — лишнее, всего 8 px).

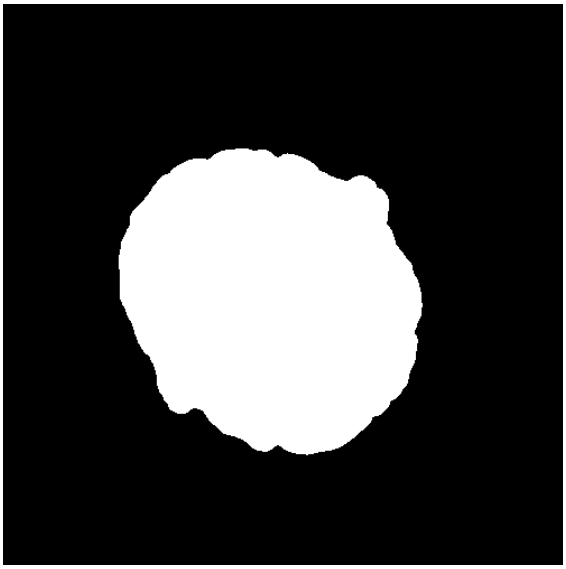
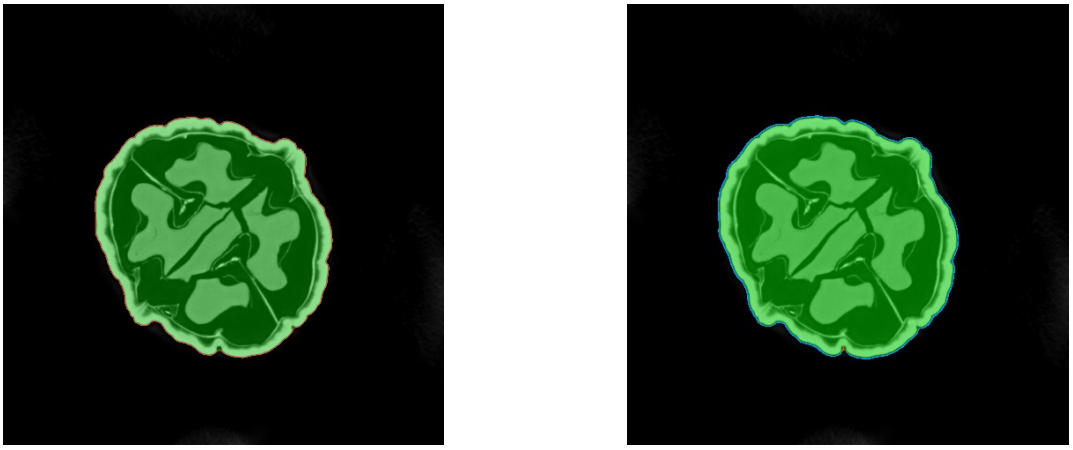


Рис. 11. Стадия A, шаг 4: итоговая маска ореха.



красное кольцо — добавлено дилатацией (+806 px) *сравнение с эталоном: TP (зел.), FN (син.), FP (красн.)*

Рис. 12. Полезность дилатации и контроль по эталону. Итоговая маска ореха: Dice = 0.989, IoU = 0.978 относительно эталонного силуэта.

3.2. Стадия B: скорлупа и внутренность (build_shell)

Функция build_shell(bl, mask, si) разделяет маску ореха на скорлупу (shell) и внутренность (interior).

Определение interior на этой стадии: interior — это всё, что внутри ореха за вычетом скорлупы, то есть (mask − shell), жёстко ограниченное внешним контуром скорлупы. Геометрически это внутреннее содержимое ореха: полости, ядро и перегородки вместе. Именно этот interior передаётся дальше в стадии C и E как область поиска. (В этих стадиях он используется по-разному — см. соответствующие разделы.)

1) Маска ореха эродируется диском 7×7 — получается суженная область me. Разность (mask − me) даёт тонкое внешнее кольцо outer, которое всегда относится к скорлупе (Рис. 13).



эрозия (me) *кольцо outer = mask − me*

Рис. 13. Стадия B: эрозия маски и выделение внешнего кольца скорлупы.

2) Внутри me выбираются пиксели исходной яркости; вычисляется перцентильный порог t (для среза 303: p = 85 → t = 176): скорлупа ярче внутреннего содержимого, поэтому верхний перцентиль яркости внутри области me соответствует внутренней стенке скорлупы. Бинаризация по этому порогу — Рис. 14.

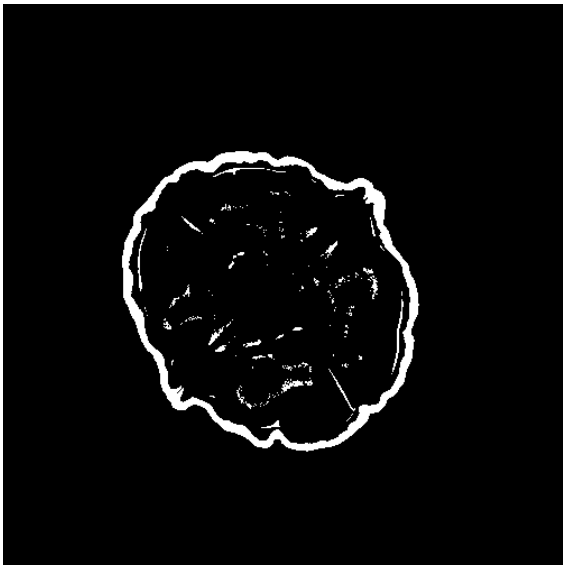
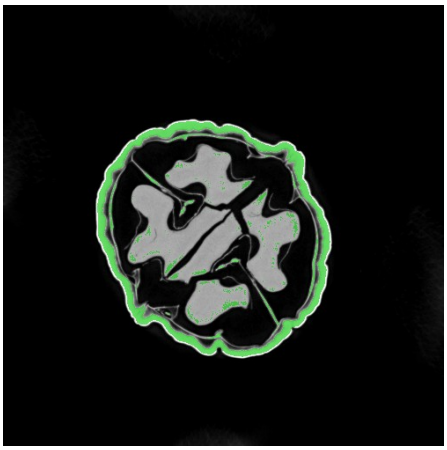


Рис. 14. Стадия B, шаг 2: бинаризация области te по перцентильному порогу (p = 85, t = 176).

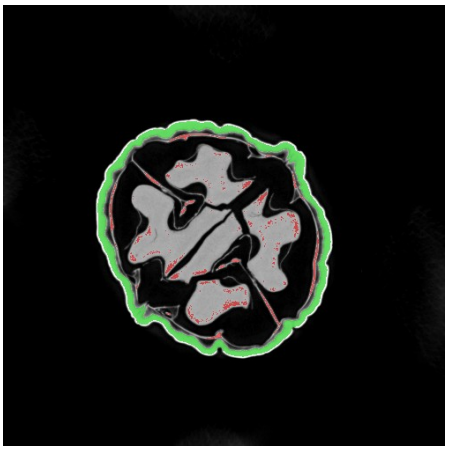
Шаг 3 — отбор компонент: принимаемые (зелёный) и отклоняемые (красный)

После порога остаётся набор связных компонент. Их нужно отфильтровать: к скорлупе относится одна крупная кольцевая компонента (внутренняя стенка), а множество мелких ярких включений внутри содержимого — это засветки на ядре и перегородках, которые требуется отбросить. Алгоритм для срезов si > 70 принимает только наибольшую по площади компоненту, остальные отклоняет.

На отладочных кадрах это показано цветом на исходном срезе: зелёным закрашены принятые пиксели, красным — отклонённые. На Рис. 15 (слева) показаны все найденные компоненты до фильтра — зелёное кольцо стенки плюс множество красноватых вкраплений внутри (яркие точки на ядре). На Рис. 15 (справа) — после фильтра: оставлена только кольцевая компонента, все внутренние включения отброшены (красным). Принятая компонента объединяется с кольцом outer — это и есть маска скорлупы.



все компоненты (зелёное — кандидаты)



после фильтра: зелёное — принято, красное — отброшено

Рис. 15. Стадия B, шаг 3: отбор компонент скорлупы. Принимается одна крупная кольцевая компонента (зелёная), внутренние яркие включения отбрасываются (красные).

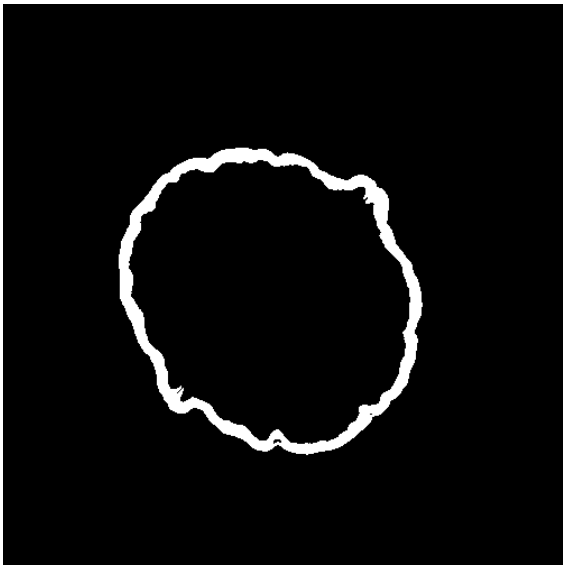


Рис. 16. Стадия B: итоговая маска скорлупы (кольцевая компонента + внешнее кольцо outer).

4) Внутренность вычисляется как (mask − shell), ограниченная внешним контуром скорлупы. Итог стадии — пара масок (shell, interior); interior показан на Рис. 17.

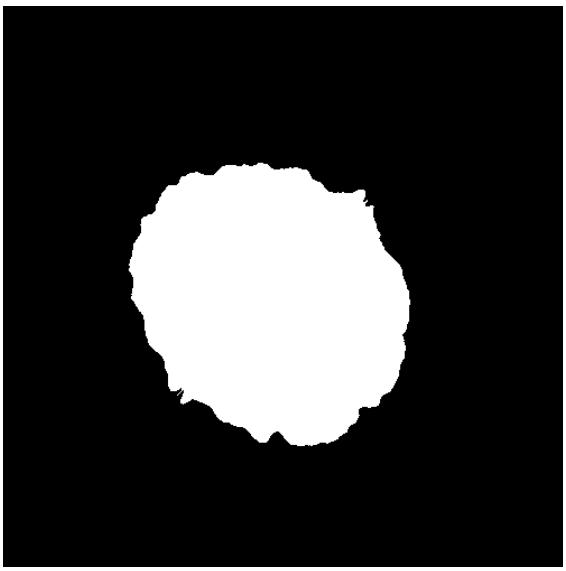


Рис. 17. Стадия B: маска внутренней области ореха (interior = mask − shell).

3.3. Стадия C: ядро (build_kernel)

Определение interior на этой стадии: здесь interior — это область поиска ядра, то есть вся начинка ореха (полученная в стадии B). Region growing запускается только внутри interior, и любой результат жёстко обрезается по нему: ядро по определению не может выйти за внутреннюю стенку скорлупы. Таким образом interior выступает «маской-ограничителем».

Функция build_kernel(bl, interior, si) активна только в диапазоне срезов [84, 383]; вне его возвращается пустая маска. Из interp_param читаются параметры: seed_pct (порог яркости для «семян»), solidity_min и aspect_min (минимальные плотность выпуклой оболочки и вытянутость).

region_growing — выращивание области

Алгоритм на основе 4-связности. Яркие пиксели interior (выше процентильного порога seed_pct) выбираются как зёрна; от каждого зерна область растёт на соседей, чья яркость отличается от текущего среднего региона не более чем на TOLERANCE. Процесс идёт, пока есть подходящие соседи. 4-связность (вместо 8-) выбрана из-за биологической природы структуры: диагональные соединения между лопастями нежелательны. Результат обрезается по interior (Рис. 18).

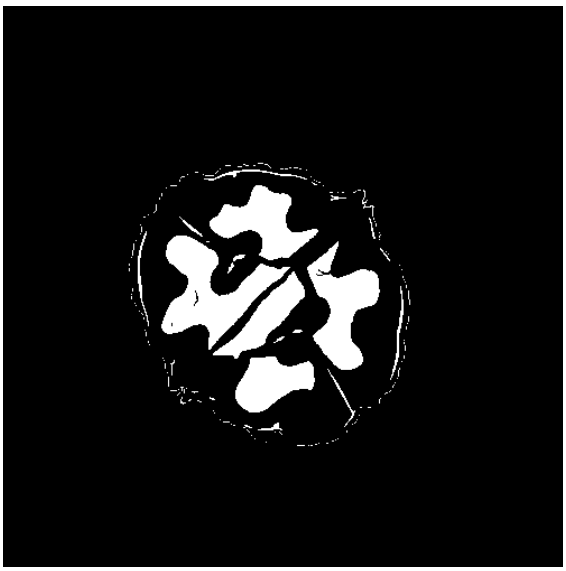


Рис. 18. Стадия C: результат region_growing (неотфильтрованное ядро до фильтрации формы).

Контроль формы: последовательность фильтров (принимаемые — зелёный, отклоняемые — красный)

Неотфильтрованный результат содержит как настоящие лопасти ядра, так и ложные компоненты (мелкие включения, тонкие протяжённые области, неплотные области). Они отсеиваются цепочкой из четырёх геометрических фильтров; на каждом отладочном кадре принятые компоненты показаны зелёным, отклонённые — красным, рядом подписана площадь a каждой компоненты.

Фильтр 1 — площадь. Компоненты меньше min_lobe = max(50, int_area × area_frac) ≈ 716 px отбрасываются как шум. На Рис. 19 крупные лопасти зелёные, мелкие точки по периметру — красные.



Рис. 19. Стадия C, фильтр 1 (площадь ≥ 716 px): крупные лопасти приняты (зелёные), мелкие компоненты отклонены (красные).

Фильтр 2 — Solidity (плотность). Solidity = площадь / площадь выпуклой оболочки. Настоящая лопасть плотная и выпуклая (Solidity высокая); неплотные, изрезанные компоненты с Solidity < solidity_min ≈ 0.36 отбрасываются (Рис. 20).



Рис. 20. Стадия C, фильтр 2 (Solidity ≥ 0.36): неплотные/изрезанные компоненты отклоняются.

Фильтр 3 — Aspect (вытянутость). Aspect = малая ось / большая ось эллипса. Лопасть округлая (Aspect не слишком мал); тонкие иглоподобные компоненты с Aspect < aspect_min ≈ 0.15 отбрасываются (Рис. 21).



Рис. 21. Стадия C, фильтр 3 (Aspect ≥ 0.15): тонкие иглоподобные компоненты отклоняются.

Фильтр 4 — доля полостей. Если внутренние полости занимают больше 20 % площади компоненты, она бракуется как ненадёжная (Рис. 22). Комбинированный результат всех четырёх фильтров — Рис. 23.



Рис. 22. Стадия C, фильтр 4 (доля полостей ≤ 20 %): компоненты с большими внутренними полостями отклоняются.



Рис. 23. Стадия C: комбинированный результат фильтрации формы (приняты только настоящие лопасти).

Итог стадии — маска ядра (Рис. 24).

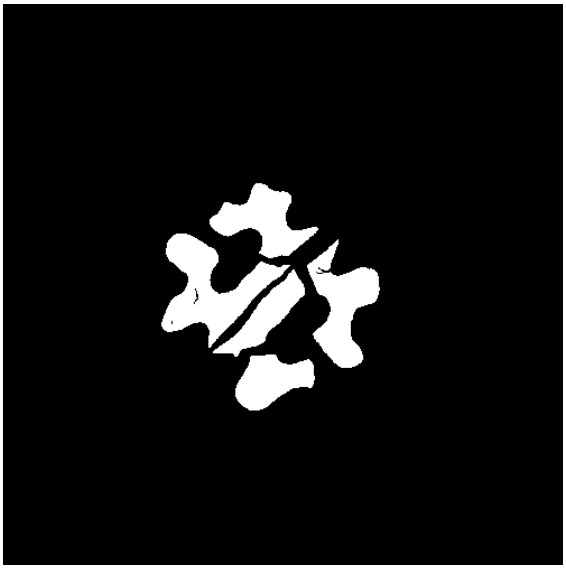


Рис. 24. Стадия C: итоговая маска ядра.

3.4. Стадия D: перегородки (build_partitions)

Определение interior на этой стадии: здесь используется не сам interior, а его эродированная версия interior_eroded = erode(interior, 3px). Смысл сужения — исключить краевую полосу шириной 3 пикселя у внутренней стенки скорлупы: на этой границе Frangi-фильтр даёт ложные срабатывания (резкий перепад яркости стенки похож на линейную структуру). Таким образом на этой стадии «рабочий interior» намеренно уже, чем на стадиях B и C.

Перегородка — это тёмная линейная структура. Перегородки в орехе неоднородны: одни проходят через центр interior, другие примыкают к ядру, третьи — к внутренней стенке скорлупы. Один метод не выявляет все типы, поэтому build_partitions объединяет три независимых источника.

Источник 1 — Frangi-фильтр

Маска interior эродируется на 3 пикселя (исключаем краевую полосу у скорлупы, где Frangi даёт ложные срабатывания). К изображению, обнулённому вне этой зоны, применяется Frangi-фильтр (black_ridges=True, sigmas = 1..3). Фильтр анализирует собственные значения матрицы Гессе: вдоль тёмного гребня кривизна почти нулевая ($\lambda_1 \approx 0$), поперёк — большая ($\lambda_2 \gg 0$); такой точке присваивается высокий отклик (Рис. 25).

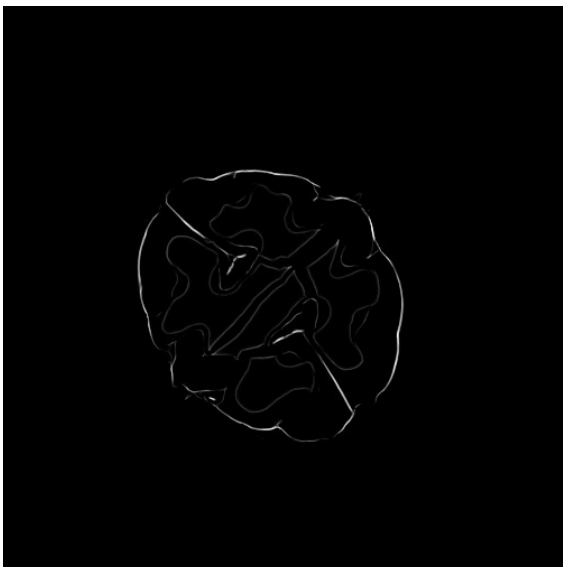


Рис. 25. Стадия D: отклик Frangi-фильтра (линейные тёмные структуры).

Отклик нормализуется и бинаризуется по порогу (оставляются верхние проценты по величине). Затем из результата вычитается расширенная маска ядра — удаляются пиксели на его контуре, которые Frangi принял за перегородки из-за резкого перепада яркости на границе (Рис. 26).

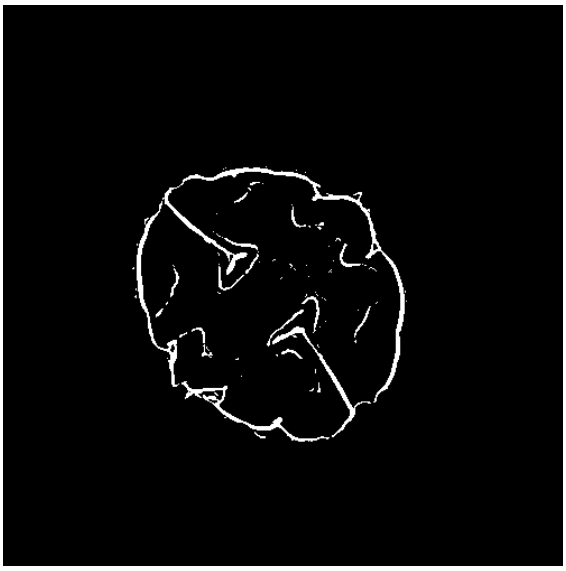


Рис. 26. Стадия D: источник 1 после пороговой обработки и удаления зоны ядра.

Источник 2 — кольцо вокруг ядра (ring bridge)

Вокруг ядра строится кольцо шириной RING_PX = 20 пикселей — именно здесь перегородки отходят от лопастей. Внутри кольца берутся пиксели ярче порога

ring_bright_pct, и оставляются только компоненты, касающиеся ядра. Это «мост» — перегородка, начинающаяся прямо у края лопасти (Рис. 27).

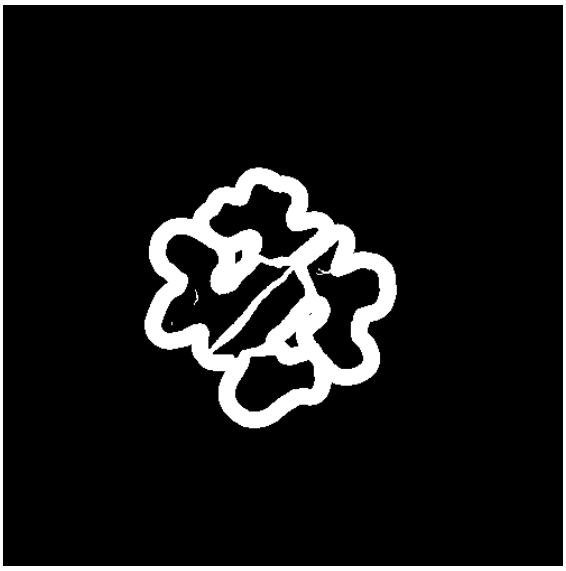
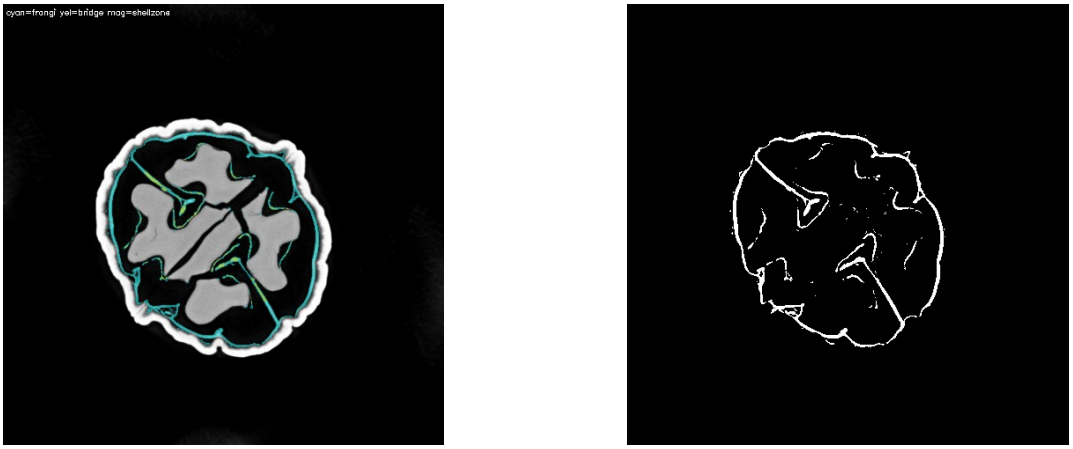


Рис. 27. Стадия D: источник 2 — кольцевая зона вокруг ядра.

Источник 3 — зона скорлупы (shell zone)

Зона между interior и скорлупой: здесь тоже бывают перегородки, примыкающие к внутренней стенке. Из неё берутся пиксели ярче фиксированного порога с фильтром по площади.

Три источника объединяются через bitwise_or (берётся всё, что хотя бы один детектор счёл перегородкой), результат обрезается по маске ореха, из него удаляется скорлупа. Цветная визуализация вкладов трёх источников и итоговая маска перегородок — Рис. 28.



вклад трёх источников (цветом) *итоговая маска перегородок*

Рис. 28. Стадия D: объединение трёх источников и итоговая маска перегородок.

3.5. Итоговый overlay среза

Функция make_overlay(r) собирает цветной контрольный overlay: серое изображение переводится в BGR, поверх него полупрозрачно (cv2.addWeighted) накладываются маски — скорлупа зелёным, ядро красным, перегородки жёлтым. Итог для среза 303 — Рис. 29.

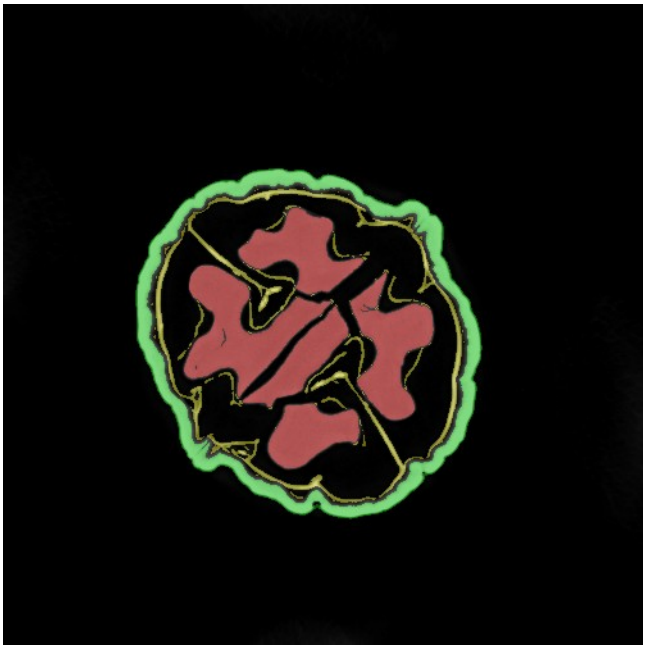


Рис. 29. Итоговый overlay среза si = 303: скорлупа (зелёный), ядро (красный), перегородки (жёлтый).

4. Оценка точности сегментации

Качество сегментации оценивалось двумя независимыми способами: метриками перекрытия с эталонными масками (Dice, IoU) и контролем плавности площадей классов по объёму.

4.1. Метрики Dice и IoU

Подготовлены 42 эталонные маски (по 14 на класс) — слайсы с 123 по 383 с шагом 20, равномерно по всему объёму. Для тех же срезов взяты маски, выданные алгоритмом. Сравнение выполнено скриптом evaluate.py, который для каждой пары считает:

```
dice = 2 * |A ∩ B| / (|A| + |B|)
iou  = |A ∩ B| / |A ∪ B|
```

Эталоны хранятся в палитровом TIFF с инвертированной палитрой (индекс 0 = белый фон, объект — тёмный), что evaluate.py корректно учитывает через ColorMap. Срез 383 для ядра исключён из статистики: на нём ядра уже нет, и его включение исказило бы среднее. Сводные результаты — в таблице ниже (получены реальным прогоном python evaluate.py).

Класс	Dice (mean ± std)	IoU (mean ± std)	Срезов
Ядро (kernel)	0.929 ± 0.016	0.869 ± 0.028	13
Скорлупа (shell)	0.885 ± 0.010	0.795 ± 0.017	14
Перегородки (septa)	0.773 ± 0.034	0.631 ± 0.044	14
Всего (41 маска)	**0.860 ± 0.070**	**0.761 ± 0.105**	**41**

Таблица 3. Метрики перекрытия с эталонными масками по классам.

Средний Dice 0.860 по всем структурам свидетельствует о хорошей точности автоматической сегментации. Наилучшее качество достигнуто для ядра (Dice 0.929) — оно крупное, плотное и контрастное. Скорлупа сегментируется стабильно (Dice 0.885, наименьший разброс). Перегородки ожидаемо сложнее всего (Dice 0.773): это тонкие структуры, где даже сдвиг на 1–2 пикселя сильно бьёт по метрике перекрытия. Высокий суммарный разброс (std 0.070 по Dice) объясняется не нестабильностью алгоритма, а принципиальной разницей структур: ядро и перегородки — объекты разной геометрической природы.

4.2. Контроль площадей по объёму (area_check.py)

Вторая, независимая от эталонов проверка опирается на физическую непрерывность: площадь каждого класса меняется вдоль оси z плавно. Резкий скачок площади на отдельном срезе — признак ошибки сегментации или артефакта. Скрипт area_check.py прогоняет пайплайн по всем срезам, собирает площадь каждого класса и строит для каждого baseline скользящей медианой (окно WINDOW = 15); срезы, где площадь отклоняется от baseline более чем на THRESHOLD (35 %), помечаются как аномальные.

График ниже (Рис. 30) построен по всем срезам рабочего объёма. Кривые площадей трёх классов гладкие, что подтверждает объёмную связность сегментации; единичные помеченные точки соответствуют переходным срезам на границах присутствия класса, а не систематическим ошибкам.

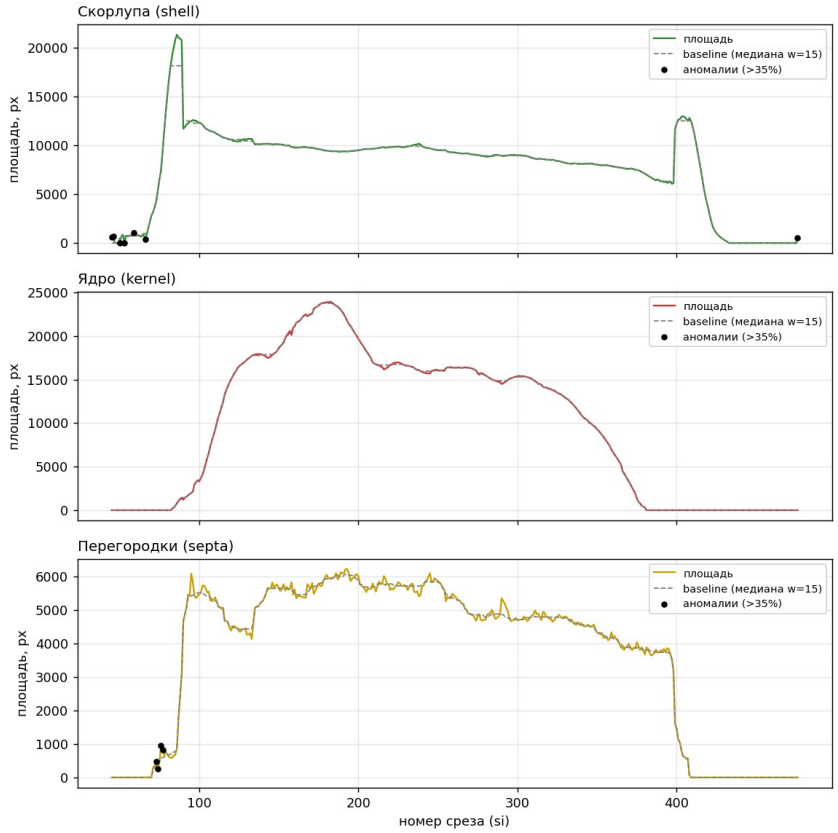


Рис. 30. Динамика площадей классов по номеру среза. Сплошная линия — площадь, пунктир — baseline (скользящая медиана, окно 15), чёрные точки — аномалии (отклонение > 35 %).

4.3. Диагностика сегментации по ортогональным проекциям (xz/yz)

Метрики §4.1–4.2 оценивают качество в плоскости одного среза, но часть дефектов видна только в объёме. Чтобы их увидеть, отдельный скрипт volume_slicer/vs.py складывает все overlay-срезы (выход этапа 1) в трёхмерный воксельный массив и берёт из него ортогональные сечения через центральную область ореха — при фиксированных координатах ≈ 239 и 259 px. Сохраняются четыре проекции (xz и yz), у которых одна ось — номер среза z; поэтому на таких разрезах сразу видна непрерывность структур по высоте. Цветовая кодировка — как на overlay: зелёный — скорлупа, красный — ядро, жёлтый — перегородки.

На вертикальных yz-разрезах (Рис. 31) скорлупа образует замкнутый гладкий контур, а основные лопасти ядра прослеживаются непрерывно; горизонтальные xz-разрезы (Рис. 32) подтверждают ту же картину в перпендикулярном направлении. Просмотр этих проекций включён в регулярный цикл диагностики: после каждой правки segm.py объём пересобирается и центральные разрезы просматриваются заново — это быстрый способ обнаружить объёмные дефекты, не различимые в плоскости одного среза.

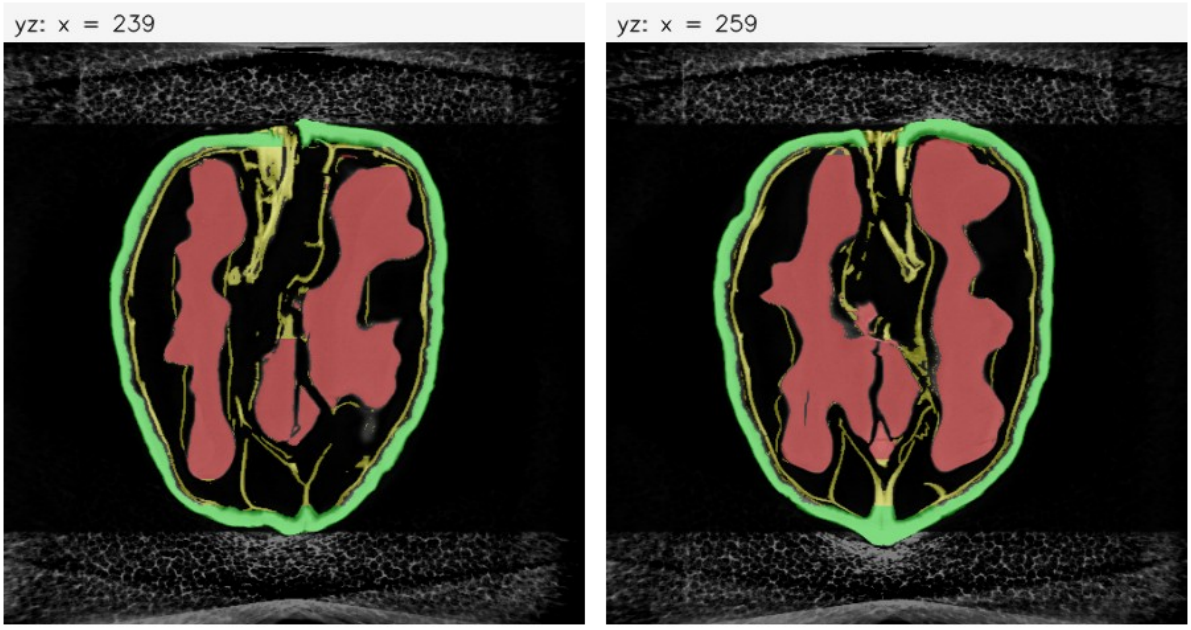


Рис. 31. Ортогональные yz-проекции объёма (vs.py) при x = 239 и x = 259. Вертикальная ось — номер среза z.

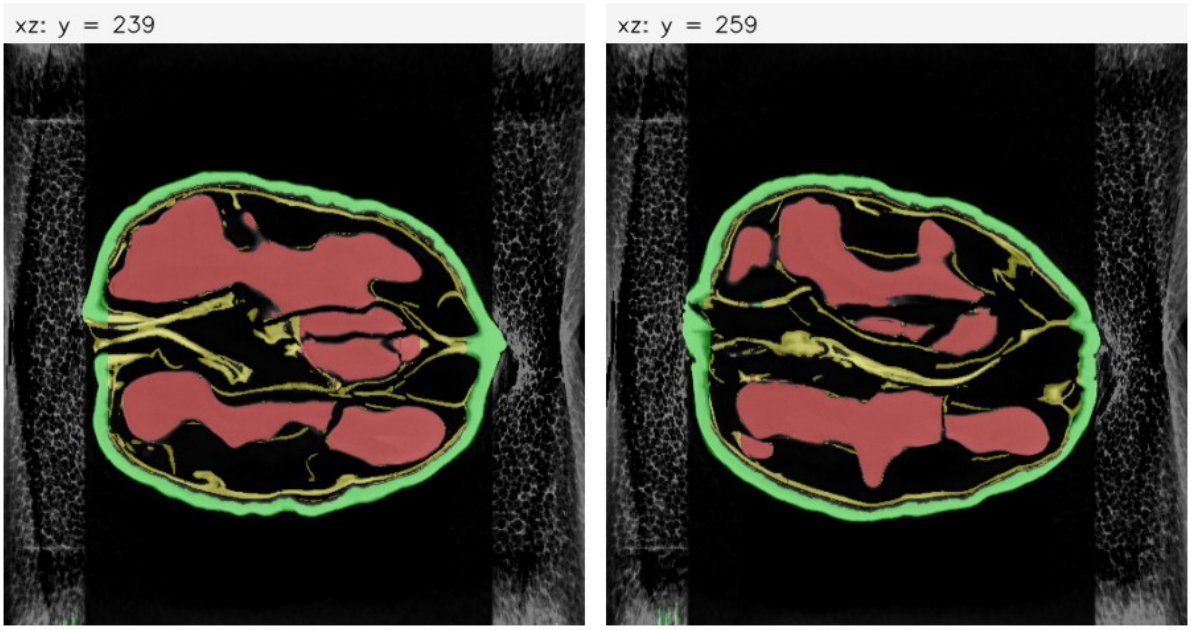


Рис. 32. Ортогональные xz-проекции объёма (vs.py) при y = 239 и y = 259 — перекрёстная проверка тех же дефектов в перпендикулярном направлении.

5. Построение 3D-мешей (build_mesh.py)

На вход второго этапа подаётся стопка overlay-изображений (результат сегментации). Скрипт build_mesh.py восстанавливает из них трёхмерный массив меток, строит для каждого класса полигональную поверхность методом Marching Cubes и сохраняет меши в OBJ и PLY. Ниже описаны метод и последовательная отладка артефактов в формате «метод → результат».

5.1. Декодирование overlay в метки (decode_labels)

Каждый пиксель overlay (в BGR) классифицируется по соотношению каналов, повторяющему цвета make_overlay: скорлупа — зелёный (G доминирует, R ≈ B), ядро — красный (R доминирует), перегородки — жёлтый (G ≈ R, оба велики, B мал).

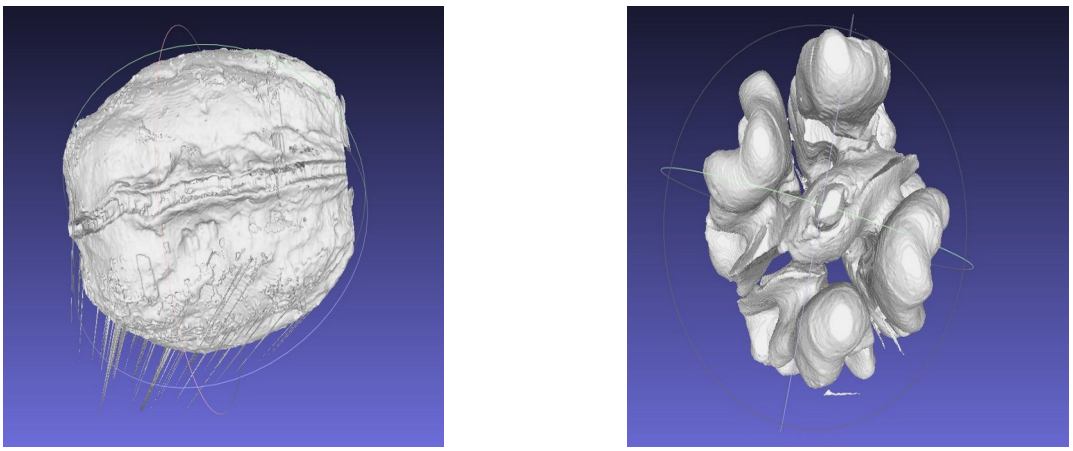
```
is_shell      = (g-r>35) & (g-b>35) & (abs(r-b)<40) & (g>40)
is_kernel     = (r-g>30) & (r-b>30) & (r>40)
is_partitions = (g>40)&(r>40)&(abs(g-r)<45)&(g-b>30) & ~is_shell & ~is_kernel
```

Срезы складываются в 3D-массив volume формы (Z, Y, X) с метками 0/1/2/3 (фон/скорлупа/ядро/перегородки).

5.2. Базовый метод и первые артефакты

Простейший пайплайн (Marching Cubes по каждому классу с лёгким сглаживанием) сразу выявил три типа артефактов. Они показаны на «сырых» результатах ниже.

Проблема 1 — остроконечные выступы у перегородок и ядра: множество тонких выступов, направленных от модели (Рис. 33).



перегородки: множество шипов *ядро: шипы слабее*

Рис. 33. Артефакты базового метода: острые выступы у перегородок и ядра.

Проблема 2 — изолированные фрагменты скорлупы: отдельные участки поверхности расположены вне основной оболочки (Рис. 34).

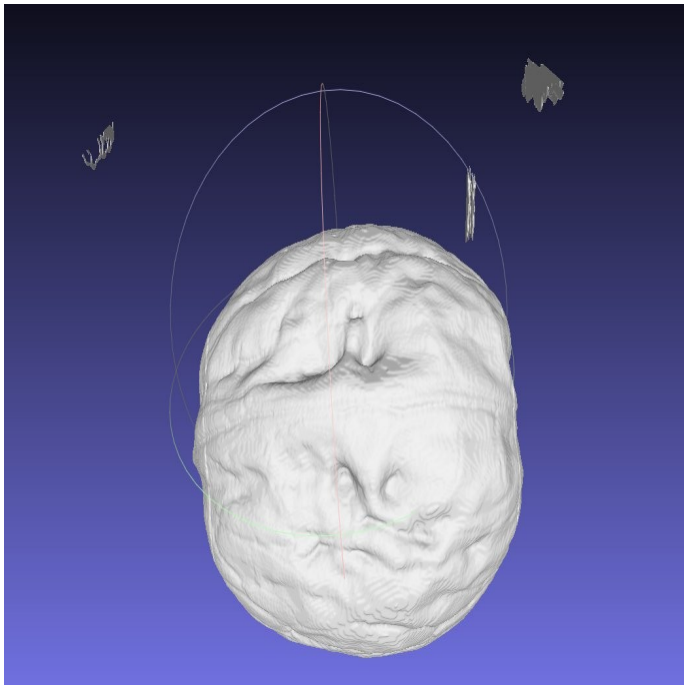


Рис. 34. Артефакт базового метода: изолированные фрагменты скорлупы вне основной оболочки.

5.3. Отладка: очистка и устранение артефактов

Причины артефактов были найдены прямым анализом объёма; для каждой реализован отдельный шаг очистки.

Шаг 1 — двухуровневая очистка компонент и обрезка краёв

Диагностика показала, что выступы порождаются мелкими изолированными пикселями на отдельных срезах: при Marching Cubes каждый такой кластер даёт острый выступ. Часть срезов скорлупы к тому же содержала пиксели только у нижнего края кадра (орех выходил за границу среза) — обрыв поверхности интерпретировался как открытый край и вытягивался в выступ. Реализованы:

- 2D-очистка каждого среза (clean_slice_2d): удаление компонент меньше MIN_COMPONENT_AREA = 100 пикселей и обнуление полосы EDGE_MARGIN = 5 пикселей вдоль краёв кадра;
- 3D-очистка (remove_small_3d_components): удаление изолированных пространственных компонент меньше MIN_3D_COMPONENT_VOXELS = 500 вокселей.

Для перегородок это сократило число 3D-компонент с 739 до 1 значимой; для скорлупы исчезли все изолированные фрагменты.

Шаг 2 — сохранение всех значимых долей (а не только наибольшей)

Первая версия очистки mesh оставляла только наибольшую связную компоненту. Для ядра это было ошибкой: ядро ореха состоит из нескольких раздельных лопастей, и все они нужны. После такой очистки (оставляющей только наибольшую компоненту) терялись целые доли (Рис. 35).

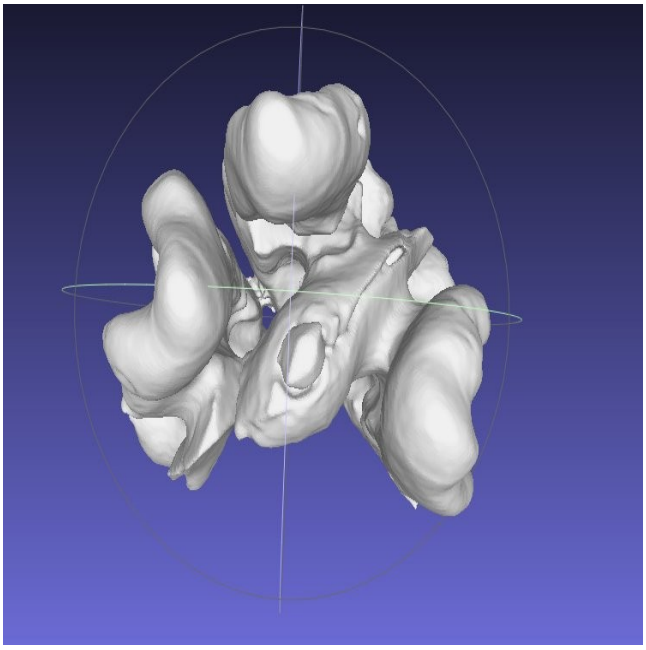


Рис. 35. Регрессия: при сохранении только наибольшей компоненты у ядра пропали целые доли.

Исправление: компонента удаляется только если её число граней меньше 1 % от наибольшей. Так отсекается лишь мусор, а все анатомические доли сохраняются.

```
components = mesh.split(only_watertight=False)
max_faces  = max(len(m.faces) for m in components)
threshold  = max(100, max_faces * 0.01)       # порог значимости
kept = [m for m in components if len(m.faces) >= threshold]
mesh = trimesh.util.concatenate(kept)
```

Шаг 3 — FloodFill только для скорлупы и ядра

На перегородках обнаружился паразитный тонкий «кольцевой» артефакт внутри модели (Рис. 36). Причина: операция заполнения полостей floodFill применялась ко всем классам. Но перегородки на ряде срезов (si ≈ 162–192) образуют почти замкнутый контур вокруг всего ореха; floodFill заполнял всё пространство внутри контура, превращая тонкие перегородки в сплошной диск, внутренняя поверхность которого и давала кольцо при Marching Cubes. На проблемных срезах заполнение добавляло до 45–48 тыс. пикселей — почти весь срез.

Исправление: floodFill применяется только к скорлупе и ядру (это сплошные объекты с реальными внутренними дырами). Для перегородок выполняется лишь лёгкое морфологическое закрытие ядром 5×5, без заполнения. Объём перегородок сократился почти вдвое (с ≈ 954 тыс. до ≈ 497 тыс. вокселей) — ровно за счёт артефактных данных.

```
is_solid = label in (LABEL_SHELL, LABEL_KERNEL)
sl = cv2.morphologyEx(sl, cv2.MORPH_CLOSE, kernel5)
if is_solid:                    # дыры заполняем только у сплошных классов
   cv2.floodFill(flooded, flood_mask, (0, 0), 1)
   sl = sl | (flooded == 0)
```

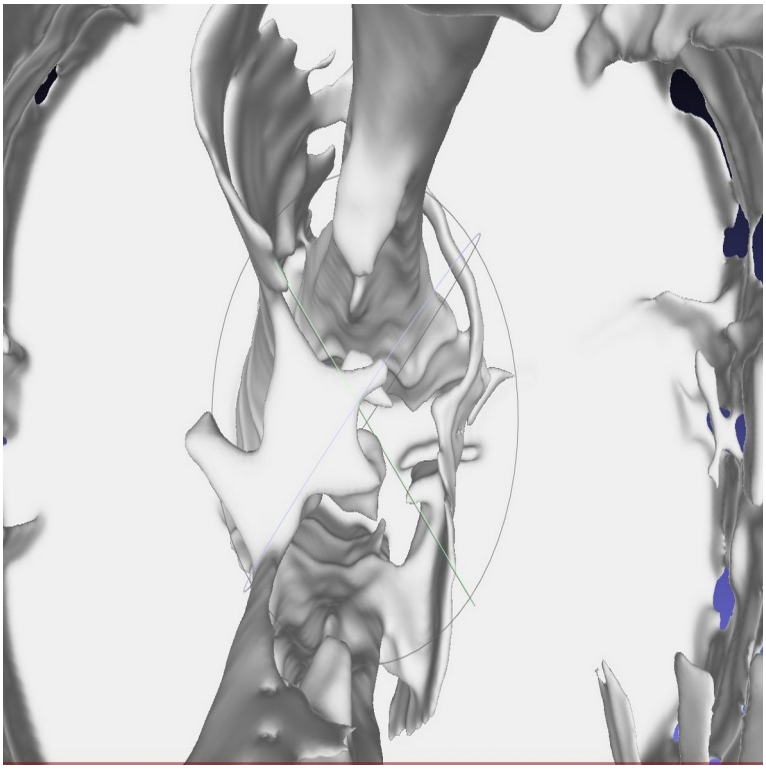


Рис. 36. Паразитное кольцо внутри перегородок (до исправления). Устранено отключением floodFill для класса перегородок.

Шаг 4 — low-poly режим (видимые полигоны)

По требованиям проекта (работа с сетками) поверхность не должна быть чрезмерно гладкой — полигоны должны быть различимы при максимальном приближении. Изначально применялись два сглаживающих шага: гауссово размытие объёма (sigma = 1.0) перед Marching Cubes и лапласово сглаживание меша (5 итераций). Оба отключены: размытие закомментировано, число итераций Лапласа по умолчанию выставлено в 0. В результате Marching Cubes выдаёт «ступенчатую» воксельную поверхность с чёткой полигональной структурой (Рис. 37).

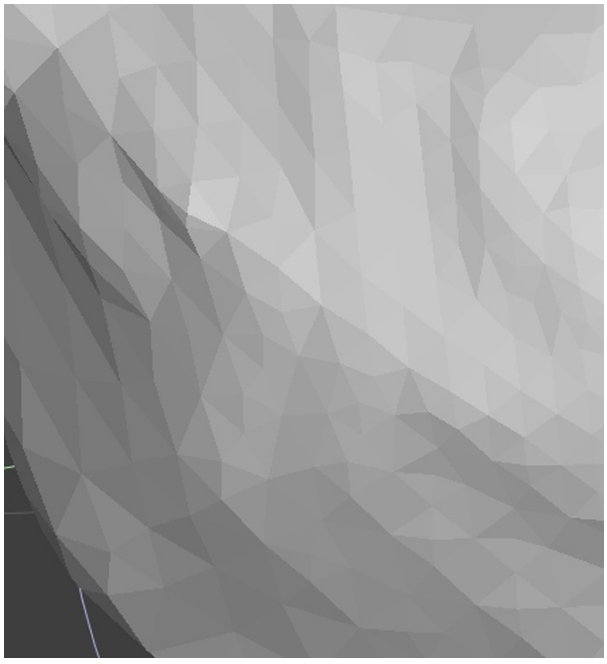


Рис. 37. Поверхность в режиме low-poly: отдельные полигоны различимы при увеличении.

Баланс между «сырым» и гладким видом регулируется параметром --smooth (число итераций Лапласа) и опциональным гауссовым размытием — оба вынесены так, чтобы их можно было включить одной строкой.

5.4. Итоговые меши

После всех шагов очистки построены три согласованных меша. Их размерности приведены в таблице ниже, визуализация — на Рис. 38–40.

Класс	Вершин	Треугольников
Скорлупа (shell)	269 620	539 236
Ядро (kernel)	302 432	605 100
Перегородки (septa)	638 372	1 296 584

Таблица 4. Размерность итоговых мешей (режим low-poly, шаг срезов = 3).



Рис. 38. Итоговый меш скорлупы.



Рис. 39. Итоговый меш ядра: раздельные лопасти сохранены.



Рис. 40. Итоговый меш перегородок (без паразитного кольца).

Параметры запуска build_mesh.py

Скрипт принимает диапазон срезов, шаг прореживания и параметры очистки. Типовые команды:

```
python build_mesh.py                    # все срезы из INPUT_DIR
python build_mesh.py 45 476 --step 3    # диапазон si 45..476, каждый 3-й срез
python build_mesh.py --smooth 1         # лёгкое сглаживание (полигоны ещё видны)
```

Ключевые константы вынесены в начало файла: MIN_COMPONENT_AREA, MIN_3D_COMPONENT_VOXELS, EDGE_MARGIN, VOXEL_SPACING. Параметр шага --step влияет на шаг по оси Z (учитывается в spacing для корректных пропорций модели).

6. Результаты

В ходе работы реализована полная цепочка от КТ-стопки до трёхмерных моделей классов ореха.

- Реализован и отлажен параметрический пайплайн сегментации segm.py с четырьмя стадиями (маска ореха → скорлупа/интерьер → ядро → перегородки). Система PARAM_ANCHORS с линейной интерполяцией обеспечивает плавную адаптацию ко всему диапазону срезов без правок логики.
- Трёхканальный детектор перегородок (Frangi + кольцо вокруг ядра + зона скорлупы) покрывает разные типы перемычек с разными механизмами контрастности.
- Расширенный debug-режим с нумерованными промежуточными изображениями делает диагностику параметров воспроизводимой и систематической; именно на нём построена пошаговая часть этого отчёта (48 кадров для среза 303).
- Качество подтверждено количественно: средний Dice 0.860 (ядро 0.929, скорлупа 0.885, перегородки 0.773) и гладкие кривые площадей по объёму.
- Реализован build_mesh.py: декодирование overlay в метки, Marching Cubes по классам, двухуровневая очистка (2D + 3D), обрезка краёв кадра, сохранение всех значимых долей ядра, раздельное заполнение полостей (floodFill только для сплошных классов) и low-poly режим с различимыми полигонами.
- Построены три согласованных меша (скорлупа, ядро, перегородки) в форматах OBJ/PLY, пригодные для дальнейшей работы с сетками.